No Symmetric Width-16 Canonical Frame for the Lifted-Product Code $C^{\mathrm{LP}}_{16,9}$

8 November 2026

Abstract

Lee asks for the canonical width f of the $[[1088, 128, 16]]$ self-adjoint lifted-product code $C^{\mathrm{LP}}_{16,9}$, for which $f \in \{16, 18\}$ is known. The question reduces to whether the anticommutation graph between the 720 weight-16 X-type and 720 weight-16 Z-type logical classes has an induced matching of size 128. We determine the full automorphism group of this finite problem: it is $C_{16} \times C_2$, generated by the cyclic shift and the $X \leftrightarrow Z$ exchange, and it has exactly three involutions; this computation and all written proofs are checked in Lean 4. We show, by exhaustive computation validated with planted positive controls and a second, differently pruned search, that no width-16 canonical frame is fixed by any of them. Each search writes a proof certificate (its full search tree), and a checker written by a different AI model from a specification alone, which regenerates the problem data itself, accepts all three certificates; for the swap class the certificate check is also proved sound in Lean. Every width-16 frame, if one exists, therefore has trivial stabilizer, and the minimum width among frames with a nontrivial symmetry is 18. We also explain the block structure of the problem (the 3^{16} frames of one quarter, and why no frame splits along the coordinate blocks), show that the largest known partial frames (88 and 84 pairs) are far from any frame, and size the remaining unrestricted search. The source question itself remains open.

1 Result and motivating question

Source	Question	Answer here
J. Y. Lee, *Designing Group-Valued Codes with Full Regular Low-Weight Bases*, arXiv:2610.06820v1, 5 Oct. 2026, Prop. 9, §IV.4, App. C.3 [1]	Is the unrestricted canonical width $f(C^{\mathrm{LP}}_{16,9})$ equal to 16 or to 18?	Open. No width-16 frame has a nontrivial symmetry of the problem (Theorem 1); any width-16 frame is asymmetric.
Project follow-up (this paper)	Which symmetry classes of frames can achieve width 16?	None (Theorems 6 and 1).

Table 1: Question, source and answer.

Lee [1] defines the canonical width $f(C)$ of a quantum code as the smallest w for which C has a canonical (symplectic) basis of logical Pauli operators of weight at most w. The source places it in the hierarchy $d \le f_{\mathrm{span}} \le f \le f_{\mathrm{reg}} \le f_{\mathrm{rep}}$ and states $f_{\mathrm{span}} = 16$ and $f_{\mathrm{reg}} = 18$ for $C^{\mathrm{LP}}_{16,9}$ with its C_{16} action. Earlier work in this repository showed $f \in \{16, 18\}$ and reduced the question to a finite matching problem (§3).

Theorem 1 (main result)**.** *No width-*16 *canonical frame of* $C^{\mathrm{LP}}_{16,9}$ *is fixed by a nontrivial element of the automorphism group* $C_{16} \times C_2$ *of the frame problem. Every width-*16 *frame, if one exists, lies in a free orbit of* 32 *frames. In particular the minimum width over frames with a nontrivial symmetry is* 18*.*

The proof is computational: Theorem 6 identifies the group, Lemma 8 reduces the claim to three involution classes, and Table 3 gives the evidence for each. The T^8 case subsumes the source's $f_{\mathrm{reg}} = 18$ and the earlier exclusion of T^4-invariant frames [5], since every nontrivial subgroup of C_{16} contains T^8.

Chronology. Table 2 lists the central sources with their dates. Elapsed days are civil calendar days to this paper's date, 8 November 2026 (Pacific); they are source ages, not the time a problem was open. Year- or month-only dates give ranges rather than an invented day. Repository records are internal manuscripts, dated by their first commit.

Table 2: Central sources and elapsed calendar days to 8 November 2026.

Source / role	Earliest relevant version; inspected milestone	Elapsed days
Lee [1]: source question (Prop. 9)	arXiv v1, 2026-10-05; v1 inspected	34
Panteleev–Kalachev [2]: lifted-product construction	arXiv v1, 2020-12-07	2,162
Repository investigation record [3] (internal)	first commit 2026-10-26	13
Repository pool enumeration [4] (internal)	first commit 2026-10-26	13
Repository T^4 exclusion [5] (internal)	first commit 2026-10-27	12
McKay–Piperno [7]: colour refinement and individualisation	arXiv v1, 2013-01-08; J. Symbolic Comput. 60, January 2014	5,052; 4,664–4,694
Knuth [8]: search-tree size estimator	Math. Comp. 29, January 1975	18,909–18,939
Heule–Kullmann–Marek [10]: cube-and-conquer with proof checking	arXiv v1, 2016-05-03	3,841
Heule [11]: Schur number five	arXiv v1, 2017-11-21	3,274

Evidence labels. [proof]: written proof. [certified]: computation that outputs an independently checkable certificate (a DRAT proof, or a search-tree certificate accepted by an independently written checker, §6). [computation]: exhaustive search by a program, cross-checked as stated, not certified. Every written proof has a Lean version (§10).

Contributions. New here: Theorem 6; Theorem 1 for the classes T^8, s and sT^8; the linear-algebra search of §6 with its validation; the structural results on blocks (Lemma 5 and §5: $Z = \pi(X)$, the 3^{16} count, no block splitting); the near-miss analysis of §7; the negative results of §8; the Lean formalization of §10, including Theorem 6; an independent implementation by a second

agent, and proof certificates for the three classes accepted by an independently written checker (§6). Cited from earlier repository work [3, 4, 5]: the pools and the reduction, the T^4 exclusion, the partial frames and the relaxation experiments.

2 Background

The code. $C^{\mathrm{LP}}_{16,9}$ is the self-adjoint lifted product [2] of the 3×5 matrix over $\mathbb{F}_4[C_{16}]$ with coefficient array $((1,1,1,1,1),(1,1,\omega,0,\omega^2),(1,\omega^2,1,\omega,0))$ and exponent array $((0,0,0,0,0),(0,6,12,9,3),(0,12,6,2,14))$ [1, eq. (50)], expanded to binary by the trace representation $\rho(\omega)=\left(\begin{smallmatrix}0&1\\1&1\end{smallmatrix}\right)$, $\rho(\omega^2)=\left(\begin{smallmatrix}1&1\\1&0\end{smallmatrix}\right)$. It has 480 independent checks of weight at most 9 in each sector, $n=1088$ and $k=128$; both CSS distances are 16 [4]. The checks are rebuilt independently by `verify_partial_frame.py`.

Definition 2. A *canonical frame* of a CSS code is a family $(\bar{X}_i,\bar{Z}_i)_{i=1}^{k}$ of logical operators in which $\bar{X}_i$ and $\bar{Z}_j$ anticommute iff $i=j$ and all other pairs commute. Its width is its largest operator weight, and $f(C)$ is the minimum width.

Coordinates and the cyclic action. Fix the width-18 frame as a basis of the logical space. Logical classes then have coordinates in $\mathbb{F}_2^{128}$, arranged as eight blocks (seeds) of 16 bits, and the shift T rotates each block by one position. With $R=\mathbb{F}_2[t]/(t^{16}+1)=\mathbb{F}_2[t]/((t+1)^{16})$, a local ring, the logical space is the free module R^8.

3 Reduction to an induced-matching problem (earlier work)

Proposition 3 ([computation]; [4])**.** *The nontrivial pure logical classes of weight at most* 17 *are exactly* 720 *X-classes and* 720 *Z-classes, all of weight* 16*, forming* 45 *free T-orbits per sector. Any canonical frame of width at most* 17 *yields a pure-sector frame drawn from these pools.*

The pool coordinates are `pools/xcoords.txt` and `pools/zcoords.txt`; index $16a+t$ is T^t applied to the a-th orbit representative. Write $G_{ij}=\langle x_i,z_j\rangle\in\mathbb{F}_2$.

Proposition 4 ([proof])**.** $f(C^{\mathrm{LP}}_{16,9})=16$ *iff there are $S,T\subseteq\{1,\dots,720\}$ with $|S|=|T|=128$ and $G[S,T]$ a permutation matrix, i.e. iff the bipartite anticommutation graph has an induced matching of size* 128*. No induced matching is larger, since its size is at most* $\operatorname{rank}_{\mathbb{F}_2}G\le128$*.*

Proof. If $G[S,T]$ is a permutation matrix then $X_SZ_T^{\mathsf{T}}$ is invertible and the matched pairs form a canonical frame; the converse is Proposition 3. An induced matching of size m is an $m\times m$ identity submatrix of G. □

Lemma 5 ([computation])**.** *G is symmetric and T-invariant, and every pool orbit spans a free rank-one R-module. Moreover $z_i=\pi(x_i)$ for every index i, where π exchanges blocks $2\leftrightarrow4$ and $3\leftrightarrow5$; so $G_{ij}=x_i^{\mathsf{T}}\pi x_j$ and a frame is a pair of bases from one pool, dual for this symmetric form. (Symmetry is asserted by `satcheck/sat_symmetric.py` and checked in Lean; the module statement is computed in §8; $z_i=\pi(x_i)$ is checked by `theory/check_pi.py`.)*

Earlier results compared against below: frames containing at least 13 seed-4 X monomials do not exist, by a scan of all 3^{16} partner assignments [3]; T^4-invariant frames do not exist (DRAT-certified) [5]; explicit partial frames with 84 and 88 pairs exist (`partial-frame-84.json`, `partial-frame-88.json`); the basic degree-two moment relaxation is feasible at 128 [5]; and an aggregate residue-descent argument fails in enlarged pools [6].

4 The symmetry group

Theorem 6 ([computation], checked in Lean)**.** *Let $\mathcal{G}$ be the group of pairs of bijections (π, π') of the index set with $G_{\pi i, \pi' j} = G_{ij}$, together with their compositions with the side exchange $s : (i, j) \mapsto (j, i)$, which preserves frames because G is symmetric. Then $\mathcal{G} = \langle T \rangle \times \langle s \rangle \cong C_{16} \times C_2$. Every linear map of the logical space preserving both pools induces an element of $\mathcal{G}$.*

Proof. Colour refinement [7] of the anticommutation graph (1440 vertices) stabilizes at the 90 T-orbits, or at 45 classes of size 32 when the two sides may be exchanged. After one vertex v is individualized, refinement reaches the discrete partition. Refinement is equivariant, so an automorphism fixing v fixes every vertex; hence $|\mathcal{G}|$ is at most the size of the colour class of v, which is 32. Both T and s are automorphisms. The computation is `explore/gram_wl.py`; coordinate permutations alone are bounded in the same way by `explore/auto_wl.py`. □

Corollary 7. *$\mathcal{G}$ has exactly three involutions, T^8, s and sT^8, and every nontrivial subgroup of $\mathcal{G}$ contains one of them.*

Lemma 8 ([proof])**.** *A frame is determined by its vertex sets (S, T). Writing τ for T^8 on indices: (i) the frame is T^8-fixed iff $\tau S = S$ and $\tau T = T$; (ii) it is s-fixed iff $S = T$; (iii) it is sT^8-fixed iff $T = \tau S$.*

Proof. The image of a frame under an element of $\mathcal{G}$ is the frame on the image vertex sets, and the induced matching on given vertex sets is unique. □

5 Evidence for Theorem 1

Class	Linear-algebra search (§6)	Independent checks
T^8	none: 9,995,481 nodes; 810,784 with look-ahead. Planted controls found (seeds 1, 2).	Certificate: 9,996,689 nodes, accepted by the independent checker (§6); supplementary: Kissat cube-and-conquer on the frozen encoding, at the time of writing 243 pieces closed and 20 open (seven hard pieces split after 55 h each), none satisfiable
s	none: 69,180 nodes, 34 s. Planted control found.	Certificate: 69,180 nodes, accepted by the independent checker, and proved in Lean (§10); supplementary: vertex SAT encoding, planted control SAT, real instance undecided after 2 h (also with symmetry breaking)
sT^8	none: 50,401,313 nodes, 2 shards, $\approx$1.85 h each. Planted control: unconstrained search exceeded its time limit (40M nodes), but with any 1 to 48 of the 64 planted items fixed the planted frame is found in at most 82 nodes and verified.	Certificate: 50,404,337 nodes in 4 shards, accepted by the independent checker
T^4 (earlier)	none: 15,088 nodes	DRAT-certified [5]

Table 3: Evidence per involution class. Programs and instance generators: `la/`, `satcheck/`; run records in `RESTRICTED-FRAMES-20261031.md`, Follow-ups E and F.

Linear-algebra search. Node counts of sharded runs include the shared top of the tree once per shard: the sT^8 search on 2 shards (50,401,313) and the certificate on 4 shards (50,404,337) both have 50,399,801 distinct nodes. The T^8 certificate is written by `la/la_gen.c` from the item instance, a different program and instance from the `la/la_dfs.c` run, so its count differs slightly. The T^8 instance is written by `la/gen_instance.py` (SHA-256 prefix `60acef5f7b8a0bf0`) and searched by `la/la_dfs.c`; the s and sT^8 instances are written by `la/gen_items.py` and searched by `la/la_gen.c`. Frames found by any run are checked by `la/verify_frame.py` or `gen_items.py verify`.

Independent SAT check. `satcheck/sat_symmetric.py` shares no code with the search. It encodes Lemma 8 directly: choose 128 X-indices; each chosen X vector has exactly one chosen Z neighbour in G and vice versa, with T fixed by the class. Kissat [9] solves it, and `satcheck/check_model.py` decodes and checks any model.

Planted controls. A random T-invariant basis and its dual basis are appended to the pools as $8+8$ extra orbits. The planted frame lies in every class, and each method must find it.

Block-diagonal frames. [computation] No frame of the 64-dimensional subspace on seeds $\{0,1,4,5\}$ or on $\{2,3,6,7\}$ uses only pool vectors supported there (instances from `la/gen_instance.py support`, searched by `la/la_dfs.c`). [certified] No frame splits along the four seed-quarters (`quarters.py`, `support_cnf.py`, DRAT proofs checked by drat-trim; `results/quarters.json`). The quarter $\{0,1\}$ alone carries exactly $3^{16} = 43{,}046{,}721$ frames of its subspace (full enumeration by the same program). [proof] This count has a short explanation: the pool vectors supported on blocks $\{0,1\}$ are $e_{0,s}$, $e_{1,s}$ and $e_{0,s}+e_{1,s}$ on both sides, so the subspace splits orthogonally into 16 copies of $\mathbb{F}_2^2$, one for each position s, each with exactly 3 frames; on the other quarters a shift by 3 between the two pools leaves no frame. [proof; computation] More generally, no frame splits along any partition of the 8 blocks: if a frame contains a full frame of the coordinates of a block set B, the rest is a frame of the other blocks, and a complete search finds a local frame only for $B \subseteq \{0,1,3,5,7\}$, whose complement contains blocks 2, 4 and 6 and so has none. Details and scripts: `theory/`.

6 Method: exact search on the residual identity

A family of 128 pairs (x_i, z_i) is a frame iff $\sum_i x_i z_i^{\mathsf{T}} = I$ over $\mathbb{F}_2$. The search state is a partial frame P together with the residual $R = I - \sum_P x_i z_i^{\mathsf{T}}$, the projection onto $C = \{z_i : i \in P\}^{\perp}$ along $\operatorname{span}\{x_i : i \in P\}$.

Lemma 9 (branching; [proof])**.** *If $R_{ck} = 1$, every completion contains a remaining pair with $x_c = z_k = 1$. Branching over those pairs, with earlier siblings excluded from later subtrees, visits each completion exactly once.*

Lemma 10 (rank pruning; [proof])**.** *Let X' (Z') be the unused pool vectors orthogonal to all chosen Z (X) vectors that belong to a still-admissible pair. A completion exists only if* $\operatorname{span} X' = C$ *and* $\operatorname{span} Z' = D$*, both of dimension* $128 - |P|$.

Lemma 11 (look-ahead; [proof])**.** *For an item with X-group a and Z-group b of size g, the child passes the rank test iff* $\operatorname{rank}(X' \cap b^{\perp}) \geq need - g$ *and* $\operatorname{rank}(Z' \cap a^{\perp}) \geq need - g$*. The pairing between these spans is then perfect automatically, so the look-ahead costs one span computation per group rather than per item.*

Validation. The general program `la/la_gen.c` reproduces the node counts of `la/la_dfs.c` exactly on the T^4 instance (15,088, with identical per-depth counts). Planted frames are found in every class tested. The quarter $\{0,1\}$ enumeration returns exactly 3^{16} frames.

Independent implementation. A separate agent (Codex, reported as based on GPT-6; exact version unavailable) wrote a second checker from a self-contained specification only (`SPEC.md` (pinned)), without access to this paper's code. It uses different methods: orbit reduction with coloured clique search, a weighted clique search for the swap classes, and a component-wise matching polynomial for counting. It passes all four known-answer tests, which we reran on our machine: planted frames found in T^8, s and sT^8; no T^4-invariant frame (complete, 25,327 nodes, 3 s); exactly 3^{16} frames on blocks $\{0,1\}$; none on blocks $\{2,3\}$. Every witness also passes our own checker. Its real T^8, s and sT^8 runs were all INCOMPLETE at their 2-hour limits (no frame found, search unfinished), so it does not confirm Theorem 1; it confirms the method-independent test answers and the T^4 result. Code and records: `independent-check/codex-gpt6/` (pinned).

Proof certificates. With option `-C`, `la/la_gen.c` writes the search tree: each node is a rank leaf, an empty leaf at a residual entry (c,k), or a branch at (c,k) listing its children. Specification: `CERT-SPEC.md`. A checker recomputes every node's state (chosen items, residual, live items) and confirms three things. The entry R_{ck} is 1. The children are exactly the live items covering (c,k), so with the residual identity and the minimal-index argument no completion is missed. Each leaf holds: an empty cover, or a live pairing matrix of rank below the remaining size. The certificates for T^8 (written from the item instance, with live-side pruning), s and sT^8 are in `certificates/`, with SHA-256 values in `certificates/MANIFEST.json`.

Independent certificate checker. ChatGPT (OpenAI; exact version not exposed by its runtime) wrote a checker in C++ from `CERT-SPEC.md` and `SPEC.md` only, without access to our search code, our reference checker or this paper. It regenerates the items from the raw pools and the class and requires them to equal the instance files. It passes the known-answer tests: it accepts the T^4 certificate, rejects three altered certificates and ten further mutations of its own, and finds and verifies the planted frame. It then accepts all three real certificates, including the cross-shard coverage checks. The node counts agree exactly with our runs: 69,180 (s, 3 s), 9,996,689 (T^8, 14 min) and 50,404,337 (sT^8, 67 min). We reran its checker on our machine with the same results. Code and records: `independent-check/chatgpt-certcheck/` (pinned). For the s class the certificate is also checked in Lean: a second checker by ChatGPT, written in Lean 4 and proved sound there, gives `no_swap_frame` (pinned): no 128-pair frame F of the real G satisfies $s(F) = F$. Its search soundness, item-table and frame-decomposition arguments are kernel-checked; two `native_decide` facts certify that the checker accepts `swap.cert.gz` and that the decoded vectors equal the pool data. We rebuilt it on our machine (`lake build`, 3,833 jobs, no errors, no `sorry`).

7 Near-miss partial frames are far from frames

Proposition 12 ([computation])**.** *Neither the* 88*-pair nor the* 84*-pair partial frame extends by a single pair. Some* 7 *of the* 88 *pairs already admit no completion. Every sampled subset of* 8 *or more pairs of the largest rank-compatible subsets (*40 *and* 41 *pairs) admits no completion.*

The driver is `la/nearmiss.py`, with fixed pairs passed to `la_gen -p`. As a control, planted frames are recovered after up to 60 of their pairs are removed. Large combinatorial partial frames

are therefore far, in shared pairs, from rank-consistent ones; rank-consistent partial frames typically fail within 3 to 8 further choices.

8 The remaining problem

Knuth's estimator [8] puts the unrestricted search at roughly 2×10^{11}–2×10^{12} nodes, with a heavy tail. At a few thousand nodes per second this is 10^4–10^6 core-hours, comparable to large SAT-based proofs [10, 11].

The following were tested and do not reduce the search materially:

- parity checks $\#\{e \in F : z_e^{\mathsf{T}} L x_e = 1\} \equiv \operatorname{tr} L$ (the smallest odd-trace check found involves at least 1515 pairs; `explore/parity_checks.py`);
- small cocircuits of the pool matroids (unit functionals, which meet 66–88 pool vectors, are near-minimal; `explore/cocircuits.py`);
- the orbit-aggregated rank-inequality LP, whose optimum is exactly 128: `explore/rank_lp.py`;
- root normalization by a forced vector, with no net gain;
- orbits forced by rank, of which there are none;
- orderly generation modulo the 32-element group: a frame without symmetry lies in a free orbit of 32 frames, so this saves at most a factor of 32 and was not implemented;
- branching first on the blocks $2, 4, 6$ where local frames fail, or on the other blocks: Knuth estimates are 1–5 times the default fewest-candidates rule (`order-exp/RESULTS.md`).

Structured pieces a frame cannot contain. [proof; computation] If a frame contains d pairs spanning (U, U'), the remaining pairs form a frame of $(U'^{\perp}, U^{\perp})$. Using this: no frame contains a whole X-orbit paired one-to-one with a whole Z-orbit (167 of the 177 such orbit pairs fail by rank, the other 10 by complete search); 41 of the 45 orbits can never appear whole in a frame on either side, and orbits $9, 10, 11, 33$ remain undecided after time-limited searches. The 88-pair partial frame contains two of the excluded orbit pairs, which explains why it cannot be completed. None of this bounds how many vectors a frame takes from each orbit, so it is not an obstruction. Details: `theory/THEORY-NOTES-2.md`.

9 Open problems

Question 13 (source question [1])**.** Is $f(C_{16,9}^{\mathrm{LP}}) = 16$? Equivalently, does the anticommutation graph contain an asymmetric induced matching of size 128?

Follow-up directions raised by this work (not source-posed):

- What is the induced matching number of the anticommutation graph? It lies between 88 and 128.
- Why do rank-consistent partial frames fail after few choices while combinatorial ones reach 88? Can this gap be turned into an obstruction?

- Is there a small k with $|F \cap X_a| \le k$ for every frame F and orbit a? With $\sum_a |F \cap X_a| = 128$ over 45 orbits, a bound $k < 3$ would exclude frames.
- Is there a symmetrization principle for canonical frames of lifted-product codes over cyclic 2-group lifts? A naive residue descent fails [6]. The local ring R and its self-dual T^8-fixed submodule $(1 + t^8)R^8$ are natural tools.
- Let the *symmetric width* be the minimum width over frames with nontrivial stabilizer. It equals 18 here. When does it differ from f, for instance for $C^{\mathrm{LP}}_{20,10}$?

10 Lean formalization

Appendix A explains the chain of central Lean proofs and prints their complete bodies, with current and commit-pinned links to each declaration. The written proofs are checked in Lean 4 (toolchain `v4.36.0-rc1`, Mathlib `5f7b4198`) in `C16Frames.lean`. All declarations compile with no `sorry`, and `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound`. The check is recorded in `verification.json`.

Paper statement	Lean declaration
Prop. 4, frame direction	`gram_one_iff_sum_outer` (pinned), `frame_of_perm_block` (pinned)
Prop. 4, size bound	`induced_matching_le_rank` (pinned), `induced_matching_le_dim` (pinned)
Lemma 8	`IsFrame` (pinned), `IsFrame.map` (pinned), `IsFrame.swap` (pinned), `fixed_swap_iff` (pinned), `fixed_half_iff` (pinned), `fixed_swapHalf_iff` (pinned)
Branching lemma (§6)	`exists_pair_of_residual_entry` (pinned), `first_member_unique` (pinned)
Rank-pruning lemma (§6)	`remaining_card_le_finrank` (pinned)
Look-ahead lemma (§6): dimension drop and perfect pairing	`perpOf` (pinned), `finrank_inf_perp_add` (pinned), `perfect_inf_perp` (pinned)
Bridge to the concrete pools (`Pools.lean`; data in `PoolData.lean`)	
Pools are T-orbits†	`X_rot` (pinned), `Z_rot` (pinned)
Lemma 5: G symmetric, T-invariant†	`G_symm` (pinned), `G_rot` (pinned)
Prop. 4 for these pools	`G_eq_mul` (pinned), `pool_matching_le_128` (pinned)
Symmetries act on pool frames†	`frame_rot` (pinned), `frame_swap` (pinned)
Lemma 8(iii) for these pools†	`tau8` (pinned), `fixed_sT8_iff` (pinned)
From the code to the pools (`Code.lean`, a port of `verify_partial_frame.py` and `audit_pools.py`)	
Code from eq. (50): 480+480 checks, weight ≤ 9, CSS†	`hx_length` (pinned), `hz_length` (pinned), `checks_weight_le_9` (pinned), `checks_commute` (pinned)
Width-18 frame is a canonical frame†	`canon_length` (pinned), `canon_logical` (pinned), `canon_gram` (pinned), `canon_width` (pinned)
Pools are distinct nontrivial weight-16 logicals with the recorded coordinates†	`pools_length` (pinned), `pools_weight_16` (pinned), `pools_logical` (pinned), `pools_distinct` (pinned), `coords_match` (pinned), `coords_nonzero` (pinned)
Physical anticommutation equals G†	`physical_pairing_eq_G` (pinned)
Mixed operators reduce to pure ones (`Mixed.lean`; Prop. 3, second sentence)	
Pairings, weights and types under the reduction	`ω_pure` (pinned), `card_supp_pure_le` (pinned), `kinds_differ_of_anticommute` (pinned), `pure_frame` (pinned)
The symmetry group (`Aut.lean`)	
Theorem 6: every automorphism is $T^k s^b$†	`aut_classification` (pinned), from `aut_of_certificate` (pinned) (refinement is equivariant, any finite graph) with `class_v0` (pinned), `dM_id` (pinned)
The 32 maps are distinct automorphisms; the graph is G†	`gmap_aut` (pinned), `gmap_distinct` (pinned), `adj_spec` (pinned)
Certificate check for the class s (ChatGPT, `CertGPT*.lean` (pinned))	
Theorem 1 for s: no s-fixed frame†	`no_swap_frame` (pinned); bridge to the (S,T) form, for any symmetric G: `no_swap_vertex_frame_of_no_pair` (pinned)

† Uses `native_decide` over the full 720×720 data, so it also trusts the Lean compiler (`Lean.ofReduceBool`). The pool data is generated from `pools/xcoords.txt` and `pools/zcoords.txt`, and the SHA-256 of both files is recorded in its header.

The abstract Lean statements: they concern any field, any pairing matrix and any finite index set. For the look-ahead lemma, Lean proves its two cores. The remaining step, that both spans are full exactly when the pairing has full rank, follows in one line because a pairing rank never

exceeds either dimension. The mixed reduction takes as a hypothesis that every operator of weight at most 17 has a stabilizer component, which follows from the computed $\mu_{XZ} = 26$ [4]. Theorem 6 is formalized for the anticommutation graph itself; its last sentence, that a linear map preserving both pools induces a graph automorphism, is immediate and not formalized. Not yet formalized: that computation and the completeness of the weight-16 enumeration [4] (every logical class of weight at most 17 lies in the pools); and the exhaustive computations, which remain [computation] or [certified] as labelled, except the s class, whose certificate is checked in Lean.

11 Reproducibility

Commands run from `restricted-frames-20261031/`; the C programs are compiled with `cc -O3 -march=native`. Search instances are regenerated by the listed command (the instances used for the certificates are committed in `certificates/`); the SHA-256 prefix (first 8 hex digits) identifies the file each recorded run used. Times are wall-clock on one core of a shared 2-core cloud machine unless stated.

Run	Command	Instance SHA-256	Result, time
T^8	`la/gen_instance.py order 2 inst/o2.txt`; `la/la_dfs -S 2 -s` i `inst/o2.txt`, $i = 0, 1$	`60acef5f`	no frame; 9,995,481 nodes; 801 s + 787 s
T^8, look-ahead	as above with `-l -f`	`60acef5f`	no frame; 810,784 nodes; 1,122 s + 1,011 s
s	`la/gen_items.py swap inst/g-swap.txt`; `la/la_gen -l inst/g-swap.txt`	`590699b0`	no frame; 69,180 nodes; 34 s
sT^8	`la/gen_items.py swapT8 inst/g-swapT8.txt`; `la/la_gen -l -S 2 -s` i `-D 1 inst/g-swapT8.txt`	`ab7d84bb`	no frame; 50,401,313 nodes; 6,679 s + 6,625 s
T^4 (validation)	`la/gen_items.py order 4 inst/g-o4.txt`; `la/la_gen inst/g-o4.txt`	`54d36d51`	no frame; 15,088 nodes; 2 s
Planted s	`la/gen_items.py --plant 1 swap inst/gp1-swap.txt`; `la/la_gen -l`	`0774c77f`	frame found, verified
Planted sT^8	`la/gen_items.py --plant 1 swapT8 inst/gp1-swapT8.txt`; `la/la_gen -l -p FIXED`	`757b7274`	frame found from 1–48 fixed items, verified
Swap SAT	`satcheck/sat_symmetric.py swap swap.cnf` [`--symbreak`]; `kissat swap.cnf`	CNF `588e61ed`; with symmetry breaking `05790b41`	UNKNOWN at 7,000 s
Certificates	`la/la_gen_c -l -C ../certificates/swap.cert.gz inst/g-swap.txt`; likewise `-S 2 -s` i `-D 1` on `inst/g-o2.txt` (T^8) and `-S 4 -s` i `-D 1` on `inst/g-swapT8.txt`	manifest	complete, no frame; 69,180 / 9,996,689 / 50,404,337 nodes; 27 s; 853 s ×2; ≈1,900 s ×4
Independent checker	`python3 run.py all` in `independent-check/chatgpt-certcheck/`	manifest	all three ACCEPT, tests pass; our rerun 3 s, 738 s, 3,675 s
Lean, s class	`cd lean && lake build CertGPTNoSwap`	manifest	`no_swap_frame`, no errors; 289 s

Run	Command	Instance SHA-256	Result, time
T^8 SAT (Mac)	`run-mac.sh`: Kissat cube-and-conquer, 4 parallel jobs on a Mac	frozen encoding	243 cubes UNSAT, 20 open
Lean	`cd lean && lake exe cache get && lake build`	in the header of `PoolData.lean`	0 errors, 3,833 jobs (with the s certificate check); `Pools` 675 s, `Code` 345 s

12 Availability

Repository links in this paper point to `master` of `https://github.com/alectionary/vibe-math`, where the work was merged with pull request 17.

Immutable review copies: the earlier work is pinned at commit `132f1161`, and the Lean files at commit `e534c654`. This paper's programs, instances and records were developed on branch `research/c16-structural-20261027T173245`, the head branch of PR 17.

References

[1] J. Y. Lee, *Designing Group-Valued Codes with Full Regular Low-Weight Bases*, arXiv:2610.06820v1, 5 October 2026, Proposition 9, Section IV.4, Appendix C.3, equation (50). `https://arxiv.org/html/2610.06820v1`. Data repository: `https://github.com/yeon-lee/public-regular-logical-frames/tree/04ad4795ad15cb89ee47b7868ff925c30b2fc731`.

[2] P. Panteleev and G. Kalachev, *Quantum LDPC codes with almost linear minimum distance*, arXiv:2012.04068. `https://arxiv.org/abs/2012.04068`.

[3] *Unrestricted canonical width of* $C^{\mathrm{LP}}_{16,9}$, investigation record. `https://github.com/alectionary/vibe-math/blob/master/formalization/canonical-width/c16/INVESTIGATION.md`.

[4] *Independent low-weight pool enumeration.* `https://github.com/alectionary/vibe-math/blob/master/formalization/canonical-width/c16/INDEPENDENT-ENUMERATION.md`.

[5] *Structural comparison: exact restricted result, no quarter-turn invariant frame.* `https://github.com/alectionary/vibe-math/blob/master/formalization/canonical-width/c16/STRUCTURAL-COMPARISON.md`.

[6] *Inverse four-hour block: enlarged Cartesian dictionaries.* `https://github.com/alectionary/vibe-math/blob/master/formalization/canonical-width/c16/INVERSE-FOUR-HOURS.md`.

[7] B. D. McKay and A. Piperno, *Practical graph isomorphism, II*, J. Symbolic Comput. 60 (2014). `https://arxiv.org/abs/1301.1493`.

[8] D. E. Knuth, *Estimating the efficiency of backtrack programs*, Math. Comp. 29 (1975). `https://www.ams.org/journals/mcom/1975-29-129/S0025-5718-1975-0373371-6/home.html`.

[9] A. Biere et al., Kissat SAT solver. `https://github.com/arminbiere/kissat`.

[10] M. J. H. Heule, O. Kullmann and V. W. Marek, *Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer*, arXiv:1605.00723. https://arxiv.org/abs/1605.00723.

[11] M. J. H. Heule, *Schur number five*, arXiv:1711.08076. https://arxiv.org/abs/1711.08076.

A Central Lean proofs

This appendix prints the complete Lean proofs on which the paper's main claims rest, generated from the checked files by `gen_lean_appendix.py`, which also checks that each declaration is identical at the pinned commit. Each name links to the current file on `master`, followed by the pinned copy. All files build with Lean 4 `v4.36.0-rc1` and Mathlib `5f7b4198` (`lake build`, 3,833 jobs, no errors, no `sorry`).

Definitions and bridges. Frames are stated twice. The paper's form is a pair (S, T) of index sets with $G[S, T]$ a permutation matrix (`IsFrame` in `C16Frames.lean`). The certificate checker uses sets F of 128 index pairs with $G_{ij'} = 1$ exactly when $(i, j) = (i', j')$ (`IsPairFrame`). The pairing is the parity of the bitwise AND of the pool vectors in `PoolData.lean`, generated from the pool files with their SHA-256 recorded in its header. `no_swap_vertex_frame_of_no_pair` carries a pair-form exclusion to the (S, T) form for any symmetric G, and `fixed_swap_iff` turns s-invariance of (S, T) into $S = T$.

Dependency structure.

- *Search lemmas.* `gram_one_iff_sum_outer` is the frame identity $\sum x_i z_i^{\mathsf{T}} = I$ behind the residual. `exists_pair_of_residual_entry` gives a covering pair at every residual entry equal to 1. `first_member_unique` justifies excluding earlier siblings. `induced_matching_le_rank` justifies rank pruning.
- *Symmetry group.* `col_iter_equiv` shows that colour refinement commutes with any automorphism. `aut_of_certificate` turns a discrete refinement after individualising one vertex into a bound on the group, for any finite graph. `class_v0` and `dM_id` are the two computed facts for the anticommutation graph, and `aut_classification` concludes that every automorphism is $T^k s^b$.
- *Swap class.* `check_sound`, with `checkChildren_sound` for the list of children, proves the generic certificate recursion sound under the obligations `RulesSound`. `completionRulesSound` discharges those obligations for the concrete frame semantics `Completion` (rank leaves, residual coverage, sibling exclusion, oversized items and the child update). `frame_root_completion` shows that every swap-invariant frame is a root completion, using the item table and the frame identity. `no_swap_frame` instantiates this with the real pools and `swap.cert.gz`; its only non-standard axioms are the two `native_decide` facts that the checker accepts the certificate and that the decoded vectors equal the pool data.

Frame identity and the search lemmas (`C16Frames.lean`)

`gram_one_iff_sum_outer` (pinned), line 27.

```
/-- Rows ‘X i‘, ‘Z i‘ with Gram matrix ‘X Zᵀ = 1‘ are dual bases: ‘Σᵢ xᵢ zᵢᵀ = Xᵀ Z = 1‘. -/
theorem gram_one_iff_sum_outer {n : ℕ} (X Z : Matrix (Fin n) (Fin n) K) :
    X * Zᵀ = 1 ↔ Xᵀ * Z = 1 := by
  constructor
  · intro h
```

```
    have h2 : Z^T * X = 1 := mul_eq_one_comm.mp h
    simpa [Matrix.transpose_mul] using congrArg Matrix.transpose h2
  · intro h
    have h2 : Z^T * X = 1 := by simpa [Matrix.transpose_mul] using congrArg Matrix.transpose h
    exact mul_eq_one_comm.mp h2
```

`exists_pair_of_residual_entry` (pinned), line 156.

```
/-- Lemma (branching): if the residual entry `R c k = Σ_{p ∈ F} x_p(c) z_p(k)` over the
remaining frame pairs `F` is `1`, some remaining pair has `x_p(c) = z_p(k) = 1`. -/
theorem exists_pair_of_residual_entry {α n : Type*} (F : Finset α) (x z : α → n → ZMod 2)
    (c k : n) (h : ∑ p ∈ F, x p c * z p k = 1) : ∃ p ∈ F, x p c = 1 ∧ z p k = 1 := by
  obtain ⟨p, hp, hne⟩ := Finset.exists_ne_zero_of_sum_ne_zero (by rw [h]; decide)
  refine ⟨p, hp, ?_, ?_⟩
  · exact zmod2_eq_one_of_ne_zero _ (left_ne_zero_of_mul hne)
  · exact zmod2_eq_one_of_ne_zero _ (right_ne_zero_of_mul hne)
```

`first_member_unique` (pinned), line 166.

```
/-- Lemma (branching), sibling exclusion: if the children `L 0, ..., L (r-1)` are tried in order
and the `u`-th subtree excludes the earlier children, a completion `A` that contains some child is
visited in exactly one subtree, namely that of the first child it contains. -/
theorem first_member_unique {α : Type*} {r : ℕ} (L : Fin r → α) (A : Set α)
    (h : ∃ u, L u ∈ A) : ∃! u, L u ∈ A ∧ ∀ v, v < u → L v ∉ A := by
  classical
  let P : ℕ → Prop := fun t => ∃ hu : t < r, L ⟨t, hu⟩ ∈ A
  have hex : ∃ t, P t := by obtain ⟨u, hu⟩ := h; exact ⟨u, u.2, hu⟩
  let t := Nat.find hex
  obtain ⟨ht, hA⟩ := Nat.find_spec hex
  refine ⟨⟨t, ht⟩, ⟨hA, ?_⟩, ?_⟩
  · intro v hv hvA
    exact Nat.find_min hex (show v.1 < t from hv) ⟨v.2, hvA⟩
  · rintro w ⟨hwA, hwmin⟩
    apply Fin.ext
    apply le_antisymm
    · by_contra hlt
      exact hwmin ⟨t, ht⟩ (not_le.mp hlt) hA
    · exact Nat.find_min' hex ⟨w.2, hwA⟩
```

`induced_matching_le_rank` (pinned), line 49.

```
/-- An induced matching of size `m` (an `m × m` identity submatrix of `G`) has `m ≤ rank G`. -/
theorem induced_matching_le_rank {m p q : ℕ} (G : Matrix (Fin p) (Fin q) K)
    (S : Fin m → Fin p) (T : Fin m → Fin q) (h : G.submatrix S T = 1) : m ≤ G.rank := by
  let Rs : Matrix (Fin m) (Fin p) K := Matrix.of fun a i => if S a = i then 1 else 0
  let Ct : Matrix (Fin q) (Fin m) K := Matrix.of fun j b => if T b = j then 1 else 0
  have hsub : Rs * G * Ct = G.submatrix S T := by
    ext a b
    simp only [Rs, Ct, Matrix.mul_apply, Matrix.of_apply, Matrix.submatrix_apply, ite_mul, one_mul,
      zero_mul, mul_ite, mul_one, mul_zero, Finset.sum_ite_eq, Finset.mem_univ, ite_true]
  have h1 : (Rs * G * Ct).rank ≤ G.rank :=
    (Matrix.rank_mul_le_left _ _).trans (Matrix.rank_mul_le_right _ _)
  rw [hsub, h, Matrix.rank_one, Fintype.card_fin] at h1
  exact h1
```

The symmetry group (`Aut.lean`)

`col_iter_equiv` (pinned), line 175.

```
theorem col_iter_equiv (k : ℕ) (a : Array ℕ) (ha : ∀ v, col a (φ v) = col a v) (v : Fin n) :
    col ((refineA G)^[k] a) (φ v) = col ((refineA G)^[k] a) v := by
  induction k generalizing v with
  | zero => exact ha v
  | succ k ih =>
    simp only [Function.iterate_succ_apply']
    exact col_refineA_equiv φ hφ _ ih v
```

`aut_of_certificate` (pinned), line 189.

```
/-- **Certificate theorem.** Suppose ‘g i‘ (with inverses ‘ginv i‘) are automorphisms, ‘K‘ rounds
of refinement put ‘v0‘ in a class consisting of images of ‘v0‘ under the ‘g i‘, and after
individualising ‘v0‘, ‘M‘ more rounds give every vertex its own index as colour. Then every
automorphism is one of the ‘g i‘. -/
theorem aut_of_certificate {ι : Type} (g ginv : ι → Fin n → Fin n)
    (hgi : ∀ i u, g i (ginv i u) = u) (hig : ∀ i u, ginv i (g i u) = u)
    (hgaut : ∀ i u w, G (g i u) (g i w) = G u w) (K M : ℕ) (v0 : Fin n)
    (hclass : ∀ u, col ((refineA G)^[K] (zeros n)) u = col ((refineA G)^[K] (zeros n)) v0 →
      ∃ i, g i v0 = u)
    (hdisc : ∀ u : Fin n, col ((refineA G)^[M] (indiv ((refineA G)^[K] (zeros n)) v0)) u = u.1)
    (φ : Equiv.Perm (Fin n)) (hφ : ∀ u w, G (φ u) (φ w) = G u w) :
    ∃ i, ∀ u, φ u = g i u := by
  have hK : ∀ ψ : Equiv.Perm (Fin n), (∀ u w, G (ψ u) (ψ w) = G u w) →
      ∀ v, col ((refineA G)^[K] (zeros n)) (ψ v) = col ((refineA G)^[K] (zeros n)) v :=
    fun ψ hψ v => col_iter_equiv ψ hψ K _ (fun v => by rw [col_zeros, col_zeros]) v
  obtain ⟨i, hi⟩ := hclass (φ v0) (hK φ hφ v0)
  -- ‘ψ = (g i)⁻¹ ∘ φ‘ is an automorphism fixing ‘v0‘.
  let ψ : Equiv.Perm (Fin n) := φ.trans ⟨ginv i, g i, fun u => hgi i u, fun u => hig i u⟩
  have hginv : ∀ u w, G (ginv i u) (ginv i w) = G u w := fun u w => by
    rw [← hgaut i, hgi, hgi]
  have hψ : ∀ u w, G (ψ u) (ψ w) = G u w := fun u w => by
    simp only [ψ, Equiv.trans_apply, Equiv.coe_fn_mk, hginv, hφ]
  have hψ0 : ψ v0 = v0 := by simp only [ψ, Equiv.trans_apply, Equiv.coe_fn_mk, ← hi, hig]
  have hd0 : ∀ v, col (indiv ((refineA G)^[K] (zeros n)) v0) (ψ v) =
      col (indiv ((refineA G)^[K] (zeros n)) v0) v := fun v => by
    rw [col_indiv, col_indiv, hK ψ hψ v]
    have : ψ v = v0 ↔ v = v0 := ⟨fun h => ψ.injective (h.trans hψ0.symm), fun h => h ▸ hψ0⟩
    simp only [this]
  have hid : ∀ u, ψ u = u := fun u => by
    have := col_iter_equiv ψ hψ M _ hd0 u
    rw [hdisc, hdisc] at this
    exact Fin.ext this
  refine ⟨i, fun u => ?_⟩
  have h := hid u
  simp only [ψ, Equiv.trans_apply, Equiv.coe_fn_mk] at h
  calc φ u = g i (ginv i (φ u)) := (hgi i _).symm
    _ = g i u := by rw [h]
```

`class_v0` (pinned), line 233.

```
/-- The colour class of vertex ‘0‘ is its orbit under the 32 known automorphisms. -/
theorem class_v0 : ∀ u : V, col cK u = col cK (0 : V) → ∃ i : Bool × Fin 16, gmap i.1 i.2 0 = u := by
  native_decide
```

`dM_id` (pinned), line 238.

```
/-- After individualising vertex ‘0‘, refinement separates all 1440 vertices: each vertex’s
colour is its own index (colours are first positions of signatures). -/
theorem dM_id : ∀ v : V, col dM v = v.1 := by native_decide
```

`aut_classification` (pinned), line 246.

```
/-- **Theorem ‘thm:aut‘.** Every automorphism of the anticommutation graph (a permutation of the
1440 vertices preserving adjacency, possibly exchanging the two sides) is one of the 32 maps
‘gmap b k = T^k ∘ s^b‘. Together with ‘gmap_aut‘ and ‘gmap_distinct‘, the automorphism group has
exactly 32 elements. -/
theorem aut_classification (φ : Equiv.Perm V) (hφ : ∀ u w, adj (φ u) (φ w) = adj u w) :
    ∃ b : Bool, ∃ k : Fin 16, ∀ u, φ u = gmap b k u := by
  obtain ⟨i, hi⟩ := aut_of_certificate adj (fun i : Bool × Fin 16 => gmap i.1 i.2)
    (fun i => ginv i.1 i.2) (fun i u => gmap_ginv i.1 i.2 u) (fun i u => ginv_gmap i.1 i.2 u)
    (fun i u w => gmap_aut i.1 i.2 u w) 3 4 0 class_v0 dM_id φ hφ
  exact ⟨i.1, i.2, hi⟩
```

Certificate soundness: the generic search kernel (`CertGPTKernel.lean`)

`RulesSound` (pinned), line 62.

```
/-- Semantic obligations for a concrete implementation. These are hypotheses,
not axioms. In particular ‘branch_cover‘ must establish residual coverage and
‘next_sound‘ must establish disjointness, orthogonality and residual updates. -/
structure RulesSound {State Item Entry Frame : Type}
    (rules : Rules State Item Entry) (Completes : State → Frame → Prop)
    (Contains : Frame → Item → Prop) : Prop where
  rank_sound : ∀ s f, Completes s f → rules.rankOk s = true → False
  empty_sound : ∀ s f e, Completes s f → rules.emptyOk s e = true → False
  branch_cover : ∀ s f e qs, Completes s f → rules.branchOk s e qs = true →
    ∃ q, q ∈ qs ∧ Contains f q
  large_sound : ∀ s f q, Completes s f → Contains f q → rules.eligible s q = true →
    rules.tooLarge s q = true → False
  next_sound : ∀ s f done q, Completes s f → Contains f q → rules.eligible s q = true →
    (∀ r, r ∈ done → ¬ Contains f r) → ∃ f', Completes (rules.next s done q) f'
```

`checkChildren_sound` (pinned), line 74.

```
private theorem checkChildren_sound {State Item Entry Frame : Type}
    (rules : Rules State Item Entry) (Completes : State → Frame → Prop)
    (Contains : Frame → Item → Prop) (sound : RulesSound rules Completes Contains)
    (run : State → Tree Item Entry → Bool)
    (run_sound : ∀ s t f, run s t = true → Completes s f → False)
    (s : State) (f : Frame) (hf : Completes s f) :
    ∀ (children : List (Item × Option (Tree Item Entry))) (done : List Item),
      (∀ r, r ∈ done → ¬ Contains f r) →
      checkChildren rules run s done children = true →
      (∃ q, q ∈ children.map Prod.fst ∧ Contains f q) → False := by
  intro children
  induction children with
  | nil =>
      intro done hd hc hex
      simp at hex
  | cons head rest ih =>
      rcases head with ⟨q, child⟩
      intro done hd hc hex
      have eligibleAndChecks := Bool.and_eq_true_iff.mp hc
      have he := eligibleAndChecks.1
      have checks := Bool.and_eq_true_iff.mp eligibleAndChecks.2
      by_cases hq : Contains f q
      · cases child with
        | none => exact sound.large_sound s f q hf hq he checks.1
        | some t =>
            obtain ⟨f', hf'⟩ := sound.next_sound s f done q hf hq he hd
            exact run_sound (rules.next s done q) t f' checks.1 hf'
      · apply ih (done ++ [q])
        · intro r hr
          rcases List.mem_append.mp hr with hr | hr
          · exact hd r hr
          · have heq : r = q := List.mem_singleton.mp hr
            simpa [heq] using hq
        · exact checks.2
        · rcases hex with ⟨r, hr, hrf⟩
          refine ⟨r, ?_, hrf⟩
          have hm : r = q ∨ r ∈ rest.map Prod.fst := by simpa using hr
          rcases hm with heq | hm
          · exact False.elim (hq (heq ▸ hrf))
          · exact hm
```

`check_sound` (pinned), line 117.

```
/-- Acceptance excludes every semantic completion, for arbitrary fuel and data.
Instantiating this theorem requires a proof of every ‘RulesSound‘ field. -/
theorem check_sound {State Item Entry Frame : Type}
    (rules : Rules State Item Entry) (Completes : State → Frame → Prop)
```

```
    (Contains : Frame → Item → Prop) (sound : RulesSound rules Completes Contains)
    (fuel : Nat) (s : State) (t : Tree Item Entry) (f : Frame)
    (accepted : check rules fuel s t = true) : ¬ Completes s f := by
  induction fuel generalizing s t f with
  | zero => simp [check] at accepted
  | succ fuel ih =>
      intro hf
      simp only [check] at accepted
      split at accepted
      · contradiction
      · cases t with
        | rank => exact sound.rank_sound s f hf accepted
        | empty e => exact sound.empty_sound s f e hf accepted
        | shard => contradiction
        | frame => contradiction
        | branch e children =>
            have checks := Bool.and_eq_true_iff.mp accepted
            exact checkChildren_sound rules Completes Contains sound (check rules fuel)
              (fun s t f hc hf => ih s t f hc hf) s f hf children []
              (by simp) checks.2 (sound.branch_cover s f e _ hf checks.1)
```

Certificate soundness: the concrete frame semantics (`CertGPTCompletion.lean`)

`Completion` (pinned), line 14.

```
/-- Concrete mathematical completion invariant. The witness is the *remaining*
item list, so choosing q passes `qs.erase q` to the child. -/
structure Completion (d : Instance) (s : SearchState) (qs : List Nat) : Prop where
  idsNodup : qs.Nodup
  live : ∀ q ∈ qs, q ∈ s.live
  itemBound : ∀ q ∈ qs, q < d.items.size
  pairBounds : ∀ p ∈ pairsOf d qs, p.1 < d.x.size ∧ p.2 < d.z.size
  pairsNodup : (pairsOf d qs).Nodup
  frame : IsPairFrame (gramField d) (pairsOf d qs).toFinset
  disjoint : ∀ q ∈ qs, ∀ r ∈ qs, q ≠ r → Disjoint (pairSet d.items[q]!) (pairSet d.items[r]!)
  count : pairCount d qs = s.need
  residual : ∀ c, c < 128 → ∀ k, k < 128 →
    bit s.residual[c]! k = paritySum (fun q => cov d q c k) qs
```

`completionRulesSound` (pinned), line 97.

```
/-- All semantic obligations for the actual executable operations are proved.
No frame-exclusion assumption or certificate acceptance is used here. -/
theorem completionRulesSound (d : Instance) :
    RulesSound (concreteRules d) (Completion d) (fun qs q => q ∈ qs) := by
  constructor
  · intro s qs h hcheck
    apply rankLeaf_contradiction d s (pairsOf d qs).toFinset h.frame
    · rw [List.toFinset_card_of_nodup h.pairsNodup, pairsOf_length, h.count]
    · intro p hp
      obtain ⟨q,hq,hp⟩ := List.mem_flatMap.mp (List.mem_toFinset.mp hp)
      exact ⟨q,h.live q hq,Array.mem_toList_iff.mp hp⟩
    · exact hcheck
  · intro s qs e h hcheck
    have hc := hcheck
    simp only [concreteRules, Bool.and_eq_true, decide_eq_true_eq] at hc
    obtain ⟨⟨⟨hc,hk⟩,_⟩,_⟩ := hc
    exact concrete_empty_contradiction d s e.1 e.2 qs h.live
      (h.residual e.1 hc e.2 hk) hcheck
  · intro s qs e children h hcheck
    have hc := hcheck
    simp only [concreteRules, Bool.and_eq_true, decide_eq_true_eq] at hc
    obtain ⟨⟨⟨hc,hk⟩,_⟩,_⟩ := hc
    exact concrete_branch_covers_remaining d s e.1 e.2 qs children h.live
      (h.residual e.1 hc e.2 hk) hcheck
  · intro s qs q h hq _ hlarge
```

```
    exact concrete_large_contradiction d s qs q hq h.count hlarge
  · intro s qs done q h hq _ hdone
    exact ⟨qs.erase q, h.next q hq done hdone⟩
```

From a swap-invariant frame to a root completion (`CertGPTRoot.lean`)

`frame_root_completion` (pinned), line 75.

```
theorem frame_root_completion (d : Instance) (F : Finset Pair)
    (hv : validateItems d "swap" = true)
    (hf : IsPairFrame (gramField d) F) (hs : F.image Prod.swap = F)
    (hb : ∀ p ∈ F, p.1 < d.x.size ∧ p.2 < d.z.size)
    (hc : F.card = 128) : ∃ qs, Completion d (rootState d) qs := by
  obtain ⟨qs, hid, hi, hn, he, hd⟩ := validated_frame_table_list d F hv hf hs hb
  have hcount : pairCount d qs = 128 := by
    rw [← pairsOf_length]
    have hh := List.toFinset_card_of_nodup hn
    rw [he, hc] at hh
    exact hh.symm
  have hfdot : IsPairFrame
      (fun i j => ∑ c : Fin 128, toVec d.x[i]! c * toVec d.z[j]! c) F := by
    have hg : (fun i j => ∑ c : Fin 128, toVec d.x[i]! c * toVec d.z[j]! c) = gramField d := by
      funext i j
      change (toVec d.x[i]! ·ᵥ toVec d.z[j]!) = bitZ (par128 (d.x[i]! &&& d.z[j]!))
      rw [par128_eq_bitParity]
      exact dot_eq_bitZ _ _
    rw [hg]
    exact hf
  have houter := pairFrame_identity (fun i => toVec d.x[i]!) (fun j => toVec d.z[j]!) F hc hfdot
  refine ⟨qs, ⟨hid, ?_, hi, ?_, hn, ?_, hd, hcount, ?_⟩⟩
  · intro q hq
    exact Array.mem_range.mpr (hi q hq)
  · intro p hp
    apply hb p
    rw [← he]
    exact List.mem_toFinset.mpr hp
  · simpa only [pairsOf, he] using hf
  · apply root_residual_of_identity d qs hn
    simpa only [he] using houter
```

`no_swap_frame_of_check` (pinned), line 109.

```
/-- Unconditional semantic soundness for bounded swap-invariant pair frames.
Both hypotheses are executable checks, not assumptions of mathematical completeness. -/
theorem no_swap_frame_of_check (d : Instance) (tokens : Array Nat)
    (itemsOK : validateItems d "swap" = true) (accepted : checkViaKernel d tokens = true) :
    ¬ ∃ F : Finset Pair,
      IsPairFrame (gramField d) F ∧ F.card = 128 ∧ F.image Prod.swap = F ∧
      (∀ p ∈ F, p.1 < d.x.size ∧ p.2 < d.z.size) := by
  rintro ⟨F, hf, hc, hs, hb⟩
  exact checkViaKernel_no_completion d tokens accepted
    (frame_root_completion d F itemsOK hf hs hb hc)
```

The concrete theorem for the swap class (`CertGPTNoSwap.lean`)

`no_finite_swap_frame_of_check` (pinned), line 22.

```
theorem no_finite_swap_frame_of_check {N : Nat} (d : Instance) (tokens : Array Nat)
    (hx : d.x.size = N) (hz : d.z.size = N)
    (hv : validateItems d "swap" = true) (ha : checkViaKernel d tokens = true) :
    ¬ ∃ F : Finset (Fin N × Fin N), IsPairFrame (arrayG d.x d.z) F ∧
      F.card = 128 ∧ F.image Prod.swap = F := by
  classical
  rintro ⟨F,hf,hc,hs⟩
```

```
  apply no_swap_frame_of_check d tokens hv ha
  refine ⟨F.image natPair, ?_, ?_, ?_, ?_⟩
  · refine ⟨?_,?_,?_⟩
    · intro p hp q hq he
      obtain ⟨a,ha,rfl⟩ := Finset.mem_image.mp hp
      obtain ⟨b,hb,rfl⟩ := Finset.mem_image.mp hq
      exact congrArg natPair (hf.1 a ha b hb (Fin.ext he))
    · intro p hp q hq he
      obtain ⟨a,ha,rfl⟩ := Finset.mem_image.mp hp
      obtain ⟨b,hb,rfl⟩ := Finset.mem_image.mp hq
      exact congrArg natPair (hf.2.1 a ha b hb (Fin.ext he))
    · intro p hp q hq
      obtain ⟨a,ha,rfl⟩ := Finset.mem_image.mp hp
      obtain ⟨b,hb,rfl⟩ := Finset.mem_image.mp hq
      have hh := hf.2.2 a ha b hb
      change arrayG d.x d.z a.1 b.2 = _
      rw [hh]
      simp only [natPair_injective.eq_iff]
  · rw [Finset.card_image_of_injective _ natPair_injective, hc]
  · calc
      (F.image natPair).image Prod.swap = (F.image Prod.swap).image natPair := by
        simp only [Finset.image_image]
        rfl
      _ = F.image natPair := by rw [hs]
  · intro p hp
    obtain ⟨a,ha,rfl⟩ := Finset.mem_image.mp hp
    exact ⟨hx ▷ a.1.isLt, hz ▷ a.2.isLt⟩
```

`no_swap_frame` (pinned), line 92.

```
/-- No size-128 swap-invariant frame exists in the actual 720×720 pool Gram matrix.
The two native_decide facts certify checking and exact equality with PoolData.
All search soundness and frame/orbit mathematics are kernel-proved. -/
theorem no_swap_frame :
    ¬ ∃ F : Finset (Fin 720 × Fin 720),
      IsPairFrame (arrayG C16Frames.Pools.xArr C16Frames.Pools.zArr) F ∧
      F.card = 128 ∧ F.image Prod.swap = F := by
  exact no_swap_from_decoded C16Frames.Pools.xArr C16Frames.Pools.zArr
    (decodeText swapEncoded) (by decide) (by decide) swap_check_ok swap_pools_ok
```

Bridge to the (S,T) form of the paper (`CertGPTVertex.lean`)

`no_swap_vertex_frame_of_no_pair` (pinned), line 88.

```
/-- Any no-swap-pair-frame result transfers to the paper's `(S,T)` formulation. -/
theorem no_swap_vertex_frame_of_no_pair {G : ι → ι → ZMod 2} {n : Nat}
    (hG : ∀ i j, G i j = G j i)
    (hno : ¬ ∃ F : Finset (ι × ι), IsPairFrame G F ∧ F.card = n ∧ F.image Prod.swap = F) :
    ¬ ∃ S T : Finset ι, C16Frames.IsFrame G S T n ∧ (T,S) = (S,T) := by
  rintro ⟨S,T,h,hfixed⟩
  exact hno (swap_fixed_isFrame_pairFrame hG h hfixed)
```